

Testdokumentation

Inhaltsverzeichnis

1. Testing-Strategie	1
1.1. Übersicht	1
1.2. Testobjekte und Teststufen	1
1.3. Testdurchführung - Entwickler	1
1.4. Testdurchführung - Integration Workflow	1
1.5. Umgang mit fehlgeschlagenen Tests	2
1.6. Manuelle Tests vor finaler Abnahme der User Storys durch den Product Owner	2
1.7. Testumfang	2
1.8. Akzeptanzkriterien	2
1.9. Umgang mit Bugs nach/während der Abnahme	2
2. Test Cases	4
2.1. Testarten	4
2.2. Testtechnologien	4
2.3. Tests	4

1. Testing-Strategie

1.1. Übersicht

Dieses Dokument beschreibt die Test-Strategie für die Anwendung. Es umfasst automatisierte Tests nach Feature-Updates sowie manuelle Tests vor der finalen Akzeptanz durch den Product Owner.

1.2. Testobjekte und Teststufen

Die Teststrategie orientiert sich an der Testpyramide: Unit-Tests bilden die Basis (Frontend und Backend), ergänzt durch manuelle Akzeptanztests vor der Abnahme. Zusätzlich werden API-Prüfungen auf Integrationsebene über Swagger durchgeführt.

- Frontend:
 - alle Komponenten werden über Unit-Tests getestet
- Backend:
 - Routen, Geschäftslogik und Authentifizierung über Unit-Tests
 - API-Prüfungen auf Integrationsebene (manuell über Swagger)

1.3. Testdurchführung - Entwickler

- Nach der Implementierung eines Features führt der verantwortliche Entwickler die Tests lokal aus
- Alle bestehenden Tests werden ausgeführt
- Es gilt das Verursacherprinzip: Der Entwickler des Features ist verantwortlich für die Testdurchführung
- Bei fehlgeschlagenen Tests muss der Entwickler die Ursache analysieren und beheben
- Es wird kein Pull Request gestellt, bevor alle Tests lokal bestanden sind
- Ebenfalls werden neue Tests für die hinzugefügten Funktionalitäten implementiert

1.4. Testdurchführung - Integration Workflow

Pull-Requests auf dev und main lösen den Integrations-Workflow aus. Dabei werden alle definierten Tests noch einmal automatisiert innerhalb eines GitHub Runners ausgeführt und das Ergebnis als Check im Pull-Request aufgeführt. Das Bestehen aller Checks ist Voraussetzung für den Merge. Checks dienen als zweite Rückfallebene, für den Fall dass ein Entwickler vergisst eine der Testroutinen lokal auszuführen und helfen dabei Testergebnisse an einer zentralen Stelle zu dokumentieren. Workflows werden im Projektverzeichnis unter [.github/workflows/](#) abgelegt.

Ein ausführliches Ablaufdiagramm ist der Entwicklerdokumentation unter dem Abschnitt *Integration und Deployment* zu entnehmen.

1.5. Umgang mit fehlgeschlagenen Tests

Bei fehlgeschlagenen Tests wird folgendes Vorgehen durchgeführt:

- **Analyse:** Ursache des Testfehlers wird untersucht.
- **Bewertung:** Entscheidung, ob
 - die neue Funktionalität korrekt ist und bestehende Tests angepasst werden müssen oder
 - die neue Funktionalität fehlerhaft ist und überarbeitet werden muss.
- **Abklärung:** Bei Bedarf Abstimmung mit Team und Product Owner.
- **Behebung:** Notwendige Anpassungen werden vorgenommen.

1.6. Manuelle Tests vor finaler Abnahme der User Storys durch den Product Owner

Vor der finalen Abnahme durch den Product Owner müssen manuelle Tests durchgeführt werden. Diese sollen die Akzeptanzkriterien der einzelnen User Storys abdecken. Die manuellen Tests entsprechen Akzeptanztests und orientieren sich an den definierten Akzeptanzkriterien der jeweiligen User Story.

1.7. Testumfang

- Simulation realistischer Benutzerszenarien
- Überprüfung der Funktionalität aus Nutzerperspektive
- Die konkreten Akzeptanztests sind in `test_cases.adoc` dokumentiert
- Validierung des Designs und der Benutzeroberfläche
- Sicherstellung der Benutzerfreundlichkeit

1.8. Akzeptanzkriterien

Eine Änderung oder ein neues Feature wird erst akzeptiert, wenn:

- Alle automatisierten Tests erfolgreich durchgeführt wurden
- Manuelle Tests erfolgreich durchgeführt wurden
- Design und Funktionalität den Anforderungen entsprechen
- Product Owner die Änderung freigegeben hat

1.9. Umgang mit Bugs nach/während der Abnahme

Sollte **während** der Abnahme ein Bug entdeckt werden, erfolgt die Dokumentation innerhalb des Pull-Requests und nicht in einem separaten Issue. Dabei wird gegebenenfalls die Funktion "Request Changes" von Github genutzt. Nach der Abnahme werden Bugs immer als Issue dokumentiert, um

den Stand der Softwarequalität nachvollziehbar zu machen.

Sollte ein Bug **nach** der Abnahme durch den Product Owner, ein Teammitglied oder externe Stakeholder entdeckt werden, wird dieser als GitHub Issue mit dem Label **bug** dokumentiert.

Alle erfassten Bugs sind zentral im GitHub-Repository einsehbar: [GitHub Issues mit Label „bug“](#)

Jedes Issue enthält eine Beschreibung des Fehlers sowie – sofern erforderlich – Screenshots oder Schritte zur Reproduktion.

2. Test Cases

Dieses Dokument beschreibt die Testfälle für die Habit Tracker Anwendung. Es umfasst alle Tests, die sicherstellen, dass die Anwendung den Anforderungen entspricht und korrekt funktioniert. Es dient als Referenz für Tester, Entwickler und Stakeholder, um die Qualität der Software zu gewährleisten.

2.1. Testarten

Wir verwenden aktuell folgende Testarten:

- **Unit Tests:** Überprüfen einzelne Funktionen und Komponenten isoliert.
- **Modul-/Handler-Tests (Backend):** Testen einzelne Express-Handler mit gestubbtten Abhängigkeiten.
- **Systemtests/Abnahmetests (manuell):** Prüfung der UI und Benutzerflüsse durch Tester.

2.2. Testtechnologien

Für die verschiedenen Testarten verwenden wir unterschiedliche Technologien:

- **Unit Tests Frontend*:** Jest + React Testing Library für Komponenten- und Utility-Tests.
- **Unit/Handler Tests Backend*:** Node.js + `assert`, Stubs/Mocks für Prisma und Supabase.
- **Systemtests (manuell)*:** Durchführung durch Tester anhand der Akzeptanzkriterien.

2.3. Tests

2.3.1. Frontend Unit Tests

`/src/frontend/tests/*.test.js` Wir haben Unit Tests für die folgenden Komponenten implementiert: * **Navbar-Komponente:** **render** **korrekt die Navigations-Links** prüft aktive Zustände für Übersicht und Hinzufügen **verlinkt auf die richtigen Pfade** zeigt Logout-Button und ruft `onLogout` auf

- **Begleiter-Komponente:**
 - zeigt je nach Stimmung (glücklich/traurig) das passende Bild
 - klick auf Begleiter ruft `onClick`-Callback auf
- **BegleiterModal-Komponente:**
 - **render** alle verfügbaren Begleiter (Katze, Hund, Wurm, Hamster)
 - Auswahl markiert Begleiter visuell und wechselbar
 - **Speichern** ruft `setAnimal` mit Token/Typ/Stimmung auf
 - **Speichern ohne Auswahl** schließt ohne API-Call
 - **Close-Button** schließt ohne zu speichern

- API-Fehler werden geloggt, onSelect wird nicht aufgerufen
- HabitCard-Komponente:
 - zeigt Habit-Name, Beschreibung und Streak
 - Kartenklick triggert onClick
 - Löschen-Button ruft onDelete mit ID auf
 - Checkbox toggelt onCheck mit ID und Zustand
 - Löschen klickt ohne Card-Click-Propagation
- HabitGrid-Komponente:
 - zeigt Kopfzeile „Bereits heute erledigt“ an
 - trennt erledigte von offenen Habits
 - Klick auf Checkbox ruft onCheck auf
 - Löschen-Button ruft onDelete auf
 - rendert auch bei leerer Liste stabil
 - zeigt Habit-Details und Streak-Informationen
- CalendarModal-Komponente:
 - zeigt Titel mit ausgewähltem Datum
 - listet erledigte und nicht erledigte Habits
 - zeigt leere Zustände bei fehlenden Einträgen
 - Close-Button ruft setOpenModal(false) auf
- Calender-Komponente:
 - rendert Monats-Header und Wochentage
 - rendert Tage des Monats
 - Navigation zu vorherigem/nächstem Monat über Buttons
 - funktioniert mit leerem Habit-Array
- HabitChangeModal-Komponente:
 - Speichern mit geändertem Namen ruft edit(id, {name, description}) auf
 - Speichern mit geänderter Beschreibung ruft edit korrekt auf
 - rendert initiale Habit-Daten
 - Speichert sowohl Änderungen bei Name und Beschreibung, wenn beide geändert wurden
- HabitCreateContent-Komponente:
 - Submit-Button deaktiviert ohne Name, aktiviert mit Name
 - Submit ruft onSubmit(name, description) auf
 - verhindert Submit, wenn der Name nur aus Leerzeilen besteht
 - erlaubt leere Beschreibung (übermittelt leeren String)
 - loggt Fehler bei fehlgeschlagenem Submit

- HabitInfoModal-Komponente:
 - zeigt Habit-Namen und optionale Beschreibung
 - zeigt Streak, Start-Datum, letztes Erledigt-Datum mit Fallbacks
 - zeigt Abschlussraten und 30-Tage-Statistik
- Calendar-Utilities (calendar.js):
 - berechnet Erledigungsgrad pro Tag
 - ermittelt erledigte und nicht erledigte Habits pro Datum
- Konvertierungs-Utilities (convert.js):
 - überprüft, dass die verschiedenen Datumsformate korrekt konvertiert werden
- Habit-Utilities (habit.js):
 - berechnet Statistiken für Habits korrekt
- API/HTTP-Utilities:
 - `httpClient` setzt Header/Body korrekt und behandelt Fehler
 - `habitsApi` ruft die richtigen Endpunkte/Methoden auf und propagiert Fehler
- Login-Komponente:
 - Submit ruft onSubmit mit E-Mail und Passwort auf
 - Register-Button ruft switchToRegister auf
- Signup-Komponente:
 - Submit ruft onSubmit mit E-Mail und Passwort auf
 - Login-Button ruft switchToLogin auf

2.3.2. Backend Unit/Handler Tests

`/src/backend/tests/*.test.js`

Wir haben Unit/Handler-Tests für folgende Backend-Bausteine implementiert:

- Prisma Habits (Handler-Logik):
 - Validierung von Input (z. B. fehlende ID/Felder)
 - Mapping/Formatierung der Response-Daten
 - Fehlerpfade (z. B. Prisma-Fehler)
- Prisma Users (Handler-Logik):
 - Validierung von Input
 - Fehlerpfade (User nicht gefunden, Prisma-Fehler)
- Routen/Handler (Auth, Habits, Users):
 - Validierung von Pflichtfeldern (z. B. Login/Logout/Session)
 - Rückgabe korrekter Statuscodes

- Supabase Auth Helper:
 - korrekte URL/Headers/Bodies
 - Fehlerbehandlung bei API-Fehlern
- Smoke Test:
 - Express-Router sind korrekt verdrahtet
 - Auth-Middleware verhält sich korrekt

2.3.3. API Integration Tests

<http://localhost:3001/api/docs/>

Die folgenden API-Endpunkte werden über Swagger manuell geprüft (Integrationstest auf API-Ebene):

- Auth:
 - POST `/api/auth/login` – Nutzeranmeldung
 - POST `/api/auth/register` – Nutzerregistrierung
 - POST `/api/auth/session` – Tokenprüfung und User-Rückgabe
 - POST `/api/auth/logout` – Abmeldung / Token-Invalidierung
- Habits:
 - GET `/api/habits` – Laden aller Habits des angemeldeten Nutzers
 - POST `/api/habits` – Anlegen eines neuen Habits
 - PUT `/api/habits/{id}` – Bearbeiten eines bestehenden Habits
 - DELETE `/api/habits/{id}` – Löschen eines Habits
 - POST `/api/habits/{id}/toggle` – Erledigt-Status für den aktuellen Tag setzen/zurücknehmen
- Users:
 - GET `/api/users/animal` – Tier-Typ und Stimmung des Nutzers abrufen
 - PUT `/api/users/animal` – Tier-Typ und/oder Stimmung setzen
- Übergreifend:
 - Prüfung geschützter Routen mittels Auth-Middleware
 - Validierung korrekter HTTP-Statuscodes und Response-Strukturen
 - Sicherstellung des Zusammenspiels von Routing, Handler-Logik und Persistenz

2.3.4. Systemtests

Akzeptanztest AT-01

Anzeige der geplanten Habits des aktuellen Tages

Ziel des Tests: Überprüfung, ob eine Übersicht aller für den aktuellen Tag geplanten Habits angezeigt wird.

Voraussetzung (Eingabe / Systemzustand): Nutzer ist eingeloggt. Für den aktuellen Tag sind Habits geplant.

Aktion (Nutzeraktion): Nutzer öffnet die Startseite der App.

Erwartetes Ergebnis (Ausgabe): Eine Liste aller für den aktuellen Tag geplanten Habits wird angezeigt.

Akzeptanztest AT-02

Visuelle Unterscheidung erledigter und offener Habits

Ziel des Tests: Überprüfung der visuellen Unterscheidbarkeit von erledigten und offenen Gewohnheiten.

Voraussetzung (Eingabe / Systemzustand): Nutzer ist eingeloggt. Es existieren erledigte und offene Habits für den aktuellen Tag.

Aktion (Nutzeraktion): Nutzer betrachtet die Habit-Übersicht.

Erwartetes Ergebnis (Ausgabe): Erledigte Gewohnheiten sind visuell eindeutig von offenen unterscheidbar.

Akzeptanztest AT-03

Sichtbarkeit der Habit-Übersicht nach Login und über Menü

Ziel des Tests: Überprüfung der Erreichbarkeit der Habit-Übersicht.

Voraussetzung (Eingabe / Systemzustand): Website der App ist erreichbar und per Browser geöffnet.

Aktion (Nutzeraktion): Nutzer betrachtet die Startseite.

Erwartetes Ergebnis (Ausgabe): Die Habit-Übersicht ist sichtbar.

Zusätzliche Aktion: Nutzer navigiert über den Menüpunkt zur Übersicht.

Erwartetes Ergebnis: Die Übersicht kann über den Menüpunkt aufgerufen werden.

Akzeptanztest AT-04

Sortierung der Gewohnheiten nach Startdatum

Ziel des Tests: Überprüfung der Sortierreihenfolge der angezeigten Gewohnheiten.

Voraussetzung (Eingabe / Systemzustand): Es existieren mehrere Gewohnheiten mit unterschiedlichen Startdaten.

Aktion (Nutzeraktion): Nutzer betrachtet die Habit-Übersicht.

Erwartetes Ergebnis (Ausgabe): Die Gewohnheiten sind nach Startdatum sortiert.

Akzeptanztest AT-05

Öffnen eines bestehenden Habits zur Bearbeitung

Ziel des Tests: Überprüfung, ob ein bestehender Habit zur Bearbeitung geöffnet werden kann.

Voraussetzung (Eingabe / Systemzustand): Nutzer ist eingeloggt. Es existiert mindestens ein Habit.

Aktion (Nutzeraktion): Nutzer wählt ein bestehendes Habit aus.

Erwartetes Ergebnis (Ausgabe): Eine Bearbeitungsansicht für den ausgewählten Habit wird angezeigt.

Akzeptanztest AT-06

Änderung von Habit-Eigenschaften

Ziel des Tests: Überprüfung, ob die Eigenschaften eines Habits geändert werden können.

Voraussetzung (Eingabe / Systemzustand): Bearbeitungsansicht eines Habits ist geöffnet.

Aktion (Nutzeraktion): Nutzer ändert mindestens eine Eigenschaft (z. B. Titel, Zeit, Dauer oder Häufigkeit) und speichert die Änderungen.

Erwartetes Ergebnis (Ausgabe): Die Änderungen werden übernommen. Die aktualisierten Werte werden angezeigt.

Akzeptanztest AT-07

Speicherung von Änderungen

Ziel des Tests: Überprüfung der dauerhaften Speicherung von Änderungen.

Voraussetzung (Eingabe / Systemzustand): Änderungen an einem Habit wurden vorgenommen.

Aktion (Nutzeraktion): Nutzer schließt die Bearbeitungsansicht und öffnet dasselbe Habit erneut.

Erwartetes Ergebnis (Ausgabe): Die zuvor vorgenommenen Änderungen sind weiterhin vorhanden.

Akzeptanztest AT-08

Feedback nach erfolgreicher Änderung

Ziel des Tests: Überprüfung, ob der Nutzer eine Rückmeldung nach erfolgreicher Speicherung erhält.

Voraussetzung (Eingabe / Systemzustand): Änderungen wurden gespeichert.

Aktion (Nutzeraktion): Nutzer beobachtet die Oberfläche nach dem Speichern.

Erwartetes Ergebnis (Ausgabe): Eine sichtbare Rückmeldung über den erfolgreichen Abschluss

wird angezeigt.

Akzeptanztest AT-09

Löschen eines Habits

Ziel des Tests: Überprüfung, ob ein Habit gelöscht werden kann.

Voraussetzung (Eingabe / Systemzustand): Nutzer ist eingeloggt. Es existiert mindestens ein Habit.

Aktion (Nutzeraktion): Nutzer löscht ein Habit.

Erwartetes Ergebnis (Ausgabe): Das Habit wird nicht mehr angezeigt und steht nicht mehr zur Bearbeitung zur Verfügung.

Akzeptanztest AT-10

„Erledigte“ Habits sind visuell erkennbar

Ziel des Tests: Überprüfung des visuellen Feedbacks nach dem Markieren eines Habits als „erledigt“.

Voraussetzung (Eingabe / Systemzustand): Ein Habit wurde erfolgreich als erledigt markiert.

Aktion (Nutzeraktion): Nutzer betrachtet die Habit-Übersicht.

Erwartetes Ergebnis (Ausgabe): Das erledigte Habit ist visuell eindeutig von nicht erledigten Habits unterscheidbar.

Akzeptanztest AT-11

Korrektur innerhalb desselben Tages

Ziel des Tests: Überprüfung der Möglichkeit zur Korrektur der Erledigt-Markierung am selben Tag.

Voraussetzung (Eingabe / Systemzustand): Nutzer befindet sich am selben Kalendertag. Ein Habit wurde zuvor als erledigt markiert.

Aktion (Nutzeraktion): Nutzer hebt die Erledigt-Markierung auf.

Erwartetes Ergebnis (Ausgabe): Der Habit gilt wieder als offen. Eine erneute Markierung ist möglich.

Akzeptanztest AT-12

Anzeige des Kalenders auf der Startseite

Ziel des Tests: Überprüfung, dass auf der Startseite eine Kalenderansicht angezeigt wird.

Voraussetzung (Eingabe / Systemzustand): Nutzer ist eingeloggt. Die Startseite wird geladen.

Aktion (Nutzeraktion): Nutzer befindet sich auf der Startseite.

Erwartetes Ergebnis (Ausgabe): Auf der Startseite wird ein Kalender angezeigt.

Akzeptanztest AT-13

Strukturierte Darstellung der Kalenderansicht

Ziel des Tests: Überprüfung, dass die Kalenderansicht Tage in einer klaren und nachvollziehbaren Struktur darstellt.

Voraussetzung (Eingabe / Systemzustand): Nutzer ist eingeloggt. Der Kalender ist sichtbar.

Aktion (Nutzeraktion): Nutzer betrachtet die Kalenderansicht.

Erwartetes Ergebnis (Ausgabe): Tage sind übersichtlich angeordnet. Die Darstellung entspricht einer klaren Struktur (z. B. Monats- oder Wochenübersicht).

Akzeptanztest AT-14

Visualisierung erledigter Habits pro Tag

Ziel des Tests: Überprüfung, dass für einzelne Tage visualisiert wird, ob Habits erledigt wurden.

Voraussetzung (Eingabe / Systemzustand): Nutzer ist eingeloggt. Es existieren Tage mit erledigten und nicht erledigten Habits.

Aktion (Nutzeraktion): Nutzer betrachtet verschiedene Tage im Kalender.

Erwartetes Ergebnis (Ausgabe): Tage mit erledigten Habits sind visuell gekennzeichnet. Tage ohne erledigte Habits sind neutral oder anders dargestellt.

Akzeptanztest AT-15

Validierung fehlerhafter Anmeldedaten

Ziel des Tests: Überprüfung der Eingabevalidierung bei falschen Zugangsdaten.

Voraussetzung (Eingabe / Systemzustand): Login-Maske ist geöffnet.

Aktion (Nutzeraktion): Nutzer gibt nicht registrierte Anmeldeinformationen ein und bestätigt die Anmeldung.

Erwartetes Ergebnis (Ausgabe): Die Anmeldung ist nicht erfolgreich. Eine verständliche Fehlermeldung wird angezeigt.

Akzeptanztest AT-16

Anzeige ausschließlich eigener Habits nach Login

Ziel des Tests: Überprüfung des Zugriffsschutzes auf persönliche Daten.

Voraussetzung (Eingabe / Systemzustand): Nutzer ist eingeloggt. Es existieren Habits mehrerer Nutzer im System.

Aktion (Nutzeraktion): Nutzer öffnet die Habit-Übersicht.

Erwartetes Ergebnis (Ausgabe): Es werden ausschließlich die eigenen Habits des Nutzers angezeigt.

Akzeptanztest AT-17

Weiterleitung auf die Startseite nach erfolgreicher Anmeldung

Ziel des Tests: Überprüfung, dass Nutzer nach erfolgreicher Anmeldung automatisch auf die Startseite weitergeleitet werden.

Voraussetzung (Eingabe / Systemzustand): Nutzerkonto existiert. Nutzer ist nicht eingeloggt.

Aktion (Nutzeraktion): Nutzer meldet sich mit gültigen Zugangsdaten an.

Erwartetes Ergebnis (Ausgabe): Die Startseite wird unmittelbar nach der Anmeldung angezeigt.

Akzeptanztest AT-18

Zugriff auf die Startseite nur für angemeldete Nutzer

Ziel des Tests: Überprüfung, dass die Startseite ausschließlich für angemeldete Nutzer zugänglich ist.

Voraussetzung (Eingabe / Systemzustand): Nutzer ist nicht eingeloggt.

Aktion (Nutzeraktion): Nutzer ruft die URL der Startseite direkt auf.

Erwartetes Ergebnis (Ausgabe): Der Nutzer wird auf die Login-Seite weitergeleitet.

Akzeptanztest AT-19

Anzeige relevanter Habits auf der Startseite

Ziel des Tests: Überprüfung, dass auf der Startseite relevante Habits angezeigt werden.

Voraussetzung (Eingabe / Systemzustand): Nutzer ist eingeloggt. Es existieren zu erledigende und erledigte Habits für den aktuellen Tag.

Aktion (Nutzeraktion): Nutzer betrachtet die Startseite.

Erwartetes Ergebnis (Ausgabe): Zu erledigende Habits werden angezeigt. Bereits erledigte Habits werden angezeigt.

Akzeptanztest AT-20

Sichtbarkeit der Navigationseinträge

Ziel des Tests: Überprüfung, dass die wichtigsten Funktionen in der Navigationsbar sichtbar sind.

Voraussetzung (Eingabe / Systemzustand): Nutzer ist eingeloggt. Anwendung ist geladen.

Aktion (Nutzeraktion): Nutzer betrachtet die Navigationsbar.

Erwartetes Ergebnis (Ausgabe): Der Reiter „Habit anlegen“ ist sichtbar. Der Reiter „Übersicht“ ist sichtbar.

Akzeptanztest AT-21

Navigation zu „Habit anlegen“

Ziel des Tests: Überprüfung, dass über die Navigationsbar zur Seite „Habit anlegen“ navigiert werden kann.

Voraussetzung (Eingabe / Systemzustand): Nutzer ist eingeloggt. Navigationsbar ist sichtbar.

Aktion (Nutzeraktion): Nutzer klickt auf den Reiter „Habit anlegen“.

Erwartetes Ergebnis (Ausgabe): Die Seite „Habit anlegen“ wird geöffnet.

Akzeptanztest AT-22

Navigation zur Übersicht der Habits

Ziel des Tests: Überprüfung, dass über die Navigationsbar zur Übersicht der Habits navigiert werden kann.

Voraussetzung (Eingabe / Systemzustand): Nutzer ist eingeloggt. Navigationsbar ist sichtbar.

Aktion (Nutzeraktion): Nutzer klickt auf den Reiter „Übersicht“.

Erwartetes Ergebnis (Ausgabe): Die Seite „Übersicht über die Habits“ (Startseite) wird geöffnet.

Akzeptanztest AT-23

Konsistentes Farbkonzept im Frontend

Ziel des Tests: Überprüfung, dass ein einheitliches Farbkonzept existiert und im gesamten Frontend angewendet wird.

Voraussetzung (Eingabe / Systemzustand): Das Frontend ist lauffähig. Ein Farbkonzept (z. B. Primär-, Sekundär- und Zustandsfarben) ist festgelegt.

Aktion (Nutzeraktion): Relevante Seiten und Ansichten des Frontends werden nacheinander geöffnet und visuell überprüft.

Erwartetes Ergebnis (Ausgabe): Farben werden konsistent gemäß dem festgelegten Farbkonzept verwendet. Es sind keine abweichenden Ad-hoc-Farben erkennbar.

Akzeptanztest AT-24

Einheitliche Benennung von CSS-Klassen

Ziel des Tests: Überprüfung, dass CSS-Klassen einem einheitlichen Benennungsschema folgen.

Voraussetzung (Eingabe / Systemzustand): Ein Benennungsschema (z. B. BEM oder teaminternes Schema) ist definiert. Der Frontend-Quellcode ist zugänglich.

Aktion (Nutzeraktion): Stichprobenartige Überprüfung von CSS/SCSS-Dateien sowie React-Komponenten.

Erwartetes Ergebnis (Ausgabe): CSS-Klassennamen entsprechen dem definierten Schema. Es sind keine uneinheitlichen oder widersprüchlichen Benennungen erkennbar.

Akzeptanztest AT-25

Einheitliche Darstellung gleicher UI-Elemente

Ziel des Tests: Überprüfung, dass gleiche UI-Elemente überall einheitlich dargestellt werden.

Voraussetzung (Eingabe / Systemzustand): Es existieren mehrfach verwendete UI-Elemente in verschiedenen Ansichten. Das Frontend ist lauffähig.

Aktion (Nutzeraktion): Gleiche UI-Elemente werden in mehreren Ansichten miteinander verglichen.

Erwartetes Ergebnis (Ausgabe): UI-Elemente besitzen identische oder klar definierte konsistente Varianten (z. B. Primary/Secondary). Abweichungen treten nur bei vorgesehenen Varianten auf.